



《AI大模型Ragent项目》第6小节：数据分块Chunk策略与实践

来自：拿个offer-开源&项目实战



马丁

2026年02月23日 23:00

为什么不能把整篇文档直接丢给大模型？

假设你在一家电商公司做开发，老板让你搞一个智能客服系统。公司有一份 200 页的《客服知识库》，涵盖退货政策、物流规则、会员权益、售后流程等内容。用户问“买了 7 天的商品还能退吗？”，系统要能从知识库里找到答案并回复。

最直觉的做法——把整份知识库的文本一股脑塞给大模型，然后让它回答。

听起来很合理，但实际跑起来你会撞上两堵墙。

1. 大模型的上下文窗口限制

大模型在处理文本时，有一个上下文窗口的概念。你可以把它理解成大模型的工作台——它一次能摊开看的纸张数量是有限的。

拿目前主流模型来说，上下文窗口大概在 128k 到 1M 个 token 之间（token 是什么后面会解释，这里你先理解成大约等于一个汉字或单个英文）。128K token 听起来很多，但一份 200 页的知识库文档，纯文本量轻松超过 30 万字，远远超出窗口上限。

直接后果：文本塞不进去，或者会触发输入长度限制/截断/费用显著增加。

从 128K token 到 1M token，跨度还不到一年，进步确实非常快。但接下来是否还需要继续把上下文做得更大，仍有待观察：从成本、延迟到实际效果来看，token 并不是越多越好。

就算未来模型的窗口越来越大，能装下整份文档了，还有第二个更本质的问题。

2. 检索精度的问题——大海捞针

假设你有一个上下文窗口无限大的模型，把整份知识库都塞进去了。用户问“生鲜商品支持七天无理由退货吗？”，模型需要从 30 万字里找到相关的那几句话。

这就像你去图书馆找一句话，但图书管理员把整个图书馆的书全摊在你面前说自己找。信息太多，噪音太大，模型很容易走神——要么找不到重点，要么把不相关的内容混进回答里。

即使上下文做到 1M，很多在线客服场景仍会偏向 RAG，因为全量长提示会显著拉高成本与响应延迟，且吞吐更差。

在 RAG 的架构中，做法不是把所有文本都丢给模型，而是先检索出最相关的几段文本，只把这几段喂给模型。这样模型拿到的上下文精准、干净，回答质量自然就上去了。

但问题来了：要做检索，你得先有可以被检索的单元。一整份知识库是没法作为检索单元的——粒度太粗了。你需要把它切成一段一段的小块，每一块聚焦一个相对完整的知识点。

这就是分块（Chunking）要干的事。

分块到底在干什么

1. 分块在 RAG 流程中的位置

上一篇我们用 Apache Tika 从 PDF、Word 等文件中提取出了纯文本。但纯文本只是原材料，还不能直接用于检索。分块是紧接着文本提取之后的一步，它把长文本切成适合检索的小段。

整个数据准备阶段的流程是这样的：



分块之后的下一步是向量化——把每个文本块转成一组数字（向量），方便计算机做相似度检索。向量化的原理和实现下一篇单独讲，这里你只需要知道：分块的质量直接决定了后续检索的质量。块切得好，检索就准；块切得烂，后面怎么优化都救不回来。

2. 几个关键参数：chunkSize、overlap

在动手切之前，有两个参数你必须搞清楚：chunkSize（块大小）和 overlap（重叠量）。

2.1 chunkSize 怎么理解

chunkSize 就是每个块的长度上限。比如你设 chunkSize = 200，意思是每个块最多包含 200 个字符（或 200 个 token，取决于你用什么单位，后面会说）。

chunkSize 设多大合适？这没有标准答案，但有一个基本的权衡：

块太大（比如 2000 字）：每个块包含的信息多，但检索时容易混入不相关的内容，精度下降。就像用户问退货政策，结果返回了一整章包含退货、换货、维修的内容，模型还得自己从里面挑。

块太小（比如 50 字）：每个块很精准，但可能把一个完整的意思切断了，上下文丢失。就像把一条退货规则从中间劈开，前半句和后半句单独看都不知道在说什么。

一般来说，200 到 1000 个字符是比较常见的范围，具体取决于你的文档类型和检索需求。

2.2 overlap 是什么，为什么需要它

overlap（重叠）是指相邻两个块之间共享的文本长度。

打个比方：你在看一本小说，每次只能记住一页的内容。如果你严格按页翻，第 1 页看完翻到第 2 页，那第 1 页最后一句话和第 2 页第一句话之间的联系就断了。但如果你每次翻页时，把上一页最后几行重新看一遍，这几行就是重叠的部分，它帮你保持了上下文的连贯性。

用一个具体的例子来看。假设知识库里有这样一段退货政策：

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。

如果 chunkSize = 40，不加 overlap，切出来可能是：

块 1：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天

块 2：无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体

块 3：以商品详情页标注为准。

注意看，“七天无理由退货”这个关键词被切成了两半，分别落在块 1 和块 2 里。用户搜“七天无理由退货”，两个块都匹配不完整。

2.2.1 不加 overlap 会丢失什么

上面的例子已经很直观了——不加 overlap，相邻块的边界处一定会丢失上下文。如果用户恰好问的问题涉及到边界处的内容，检索就可能找不到完整的答案。

加上 overlap = 15 之后，切出来变成：

块 1：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天

块 2：消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适

块 3：特殊品类不适用此规则，具体以商品详情页标注为准。

块 2 的开头和块 1 的结尾有重叠，“七天无理由退货”在块 2 里是完整的了。

当然，overlap 也不是越大越好。overlap 太大意味着大量重复文本，存储和计算成本都会上升。一般经验是 overlap 设为 chunkSize 的 10%~25%。

2.3 用什么单位：字符 vs token

你可能注意到了，前面说 chunkSize 的时候，有时说字符，有时说 token。这两个东西不一样，有必要区分一下。

字符（Character）就是你肉眼看到的每一个符号。“你好”是 2 个字符，“Hello”是 5 个字符，空格和标点也各算 1 个字符。

Token 是大模型实际处理文本的最小单位。大模型不是一个字一个字地读文本的，它会先把文本切成一个个 token。对于英文，一个常见单词通常是 1 个 token，长一点的单词可能被拆成 2-3 个 token。对于中文，一个汉字通常是 1-2 个 token。

举个例子：

文本	字符数	大约 token 数
七天无理由退货	7	7-10
Hello World	11	2
退货政策	4	4-6

为什么要关心这个？因为大模型的上下文窗口是按 token 算的，不是按字符算的。如果你用字符数来设 chunkSize，实际消耗的 token 数可能比你预期的多（尤其是中文场景）。

不过在入门阶段，用字符数来设 chunkSize 完全够用。等你对 token 有了更深的理解，再考虑切换到基于 token 的分块也不迟。

主流分块策略详解

下面我们用同一段示例文本，来演示几种主流的分块策略。先把示例文本贴出来，后面反复会用到：

一、退货政策

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在 7 个工作日内发货，运费由消费者承担。

三、会员权益

普通会员享受 9.5 折优惠，每月可领取 2 张满 200 减 20 的优惠券。黄金会员享受 9 折优惠，每月可领取 4 张满 200 减 30 的优惠券。铂金会员享受 8.5 折优惠，每月可领取 6 张满 200 减 40 的优惠券。

四、物流配送

默认使用顺丰快递，偏远地区使用邮政 EMS。订单金额满 99 元包邮，未满 99 元收取 8 元运费。大件商品（家具、家电等）使用德邦物流，运费另议。

五、售后服务

所有商品享受 1 年质保服务。质保期内出现非人为损坏的质量问题，可免费维修或更换。超出质保期的维修服务按成本价收取配件费及人工费。

这段文本大约 600 个字符，包含 5 个章节，每个章节讲一个独立的业务规则。接下来看看不同的分块策略会把它切成什么样。

1. 固定大小分块（Fixed Size Chunking）

1.1 原理

这是最简单粗暴的方式：不管文本内容是什么，每隔固定数量的字符就切一刀。

假设 chunkSize = 100，overlap = 0，上面那段示例文本会被这样切：

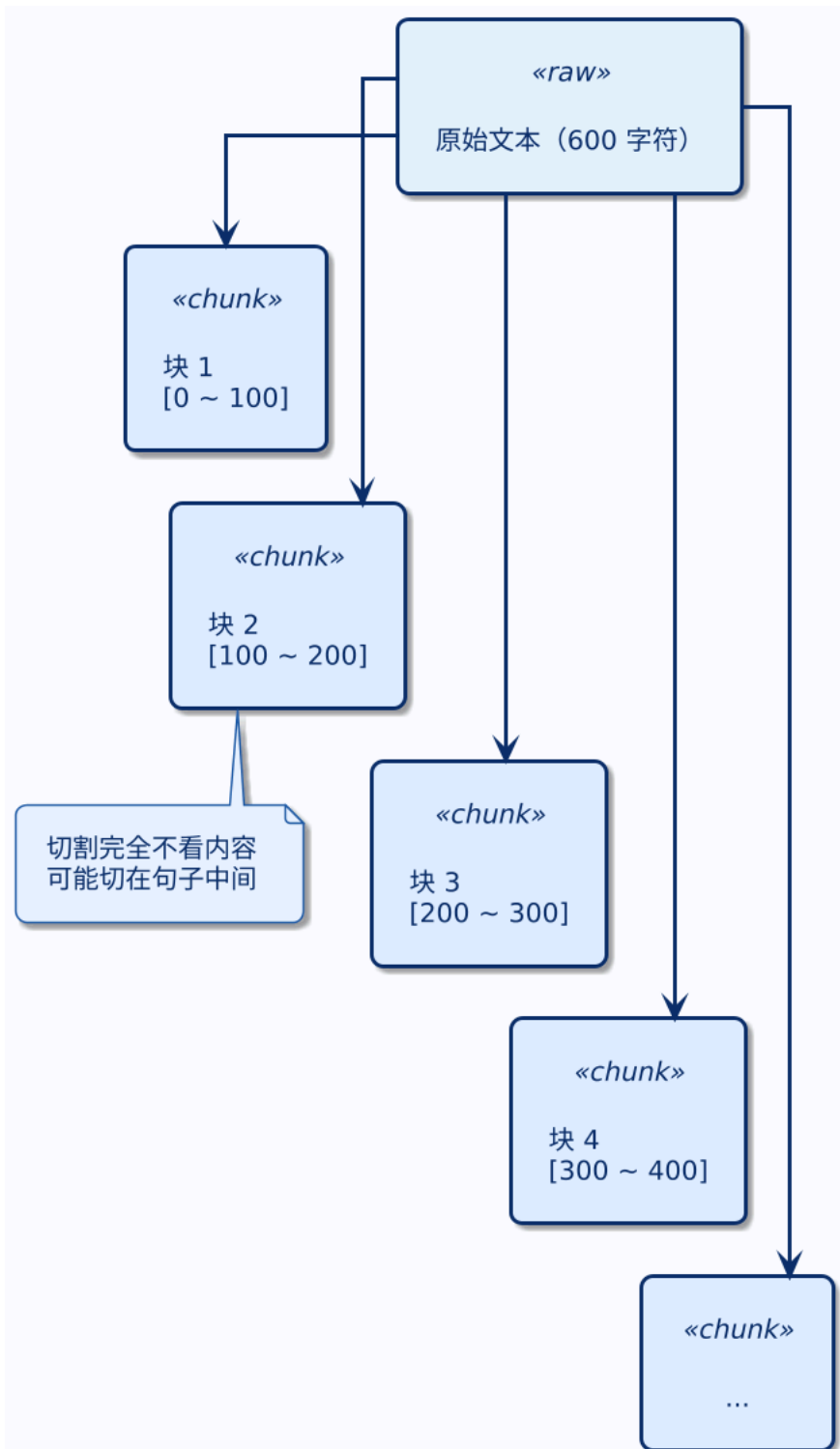
块 1: 从第 1 个字符开始, 取 100 个字符

块 2: 从第 101 个字符开始, 再取 100 个字符

块 3: 从第 201 个字符开始, 再取 100 个字符

.....依此类推

用一张图来理解:



固定大小分块完全不关心文本的语义——它不管你这一刀切在段落中间还是句子中间, 到了字数就切。这既是它的优点 (简单), 也是它的缺点 (可能切断语义)。

1.2 Java 代码实现

```
import java.util.ArrayList;
import java.util.List;

public class FixedSizeChunker {

    /**
```

```
* 固定大小分块—最基础的分块方式，按字符数硬切
*/
public static List<String> chunk(String text, int chunkSize) {
    List<String> chunks = new ArrayList<>();
    int start = 0;
    while (start < text.length()) {
        int end = Math.min(start + chunkSize, text.length());
        chunks.add(text.substring(start, end));
        start = end;
    }
    return chunks;
}

public static void main(String[] args) {
    String text = "自签收之日起 7 天内，商品未经使用且不影响二次销售的，"
        + "消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等"
        + "特殊品类不适用此规则，具体以商品详情页标注为准。"
        + "退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。";

    List<String> chunks = chunk(text, 40);

    for (int i = 0; i < chunks.size(); i++) {
        System.out.println("=== 块 " + (i + 1) + " ===");
        System.out.println(chunks.get(i));
        System.out.println();
    }
}
```

跑一下这段代码，你会看到每个块都是严格的 40 个字符，切割位置完全不考虑句子边界。有些块的开头可能是上一句话的尾巴，读起来很突兀。

1.3 优缺点

维度	说明
优点	实现极其简单，性能好，不需要任何 NLP 处理
缺点	完全忽略文本结构，容易把句子、段落从中间切断，导致语义不完整
适合	文本结构不重要的场景，比如日志文件、纯数据文本；或者作为其他策略的兜底方案
不适合	有明确段落结构的文档（知识库、产品手册、政策文件），因为切断语义会严重影响检索质量

2. 重叠分块（Overlapping Chunking）

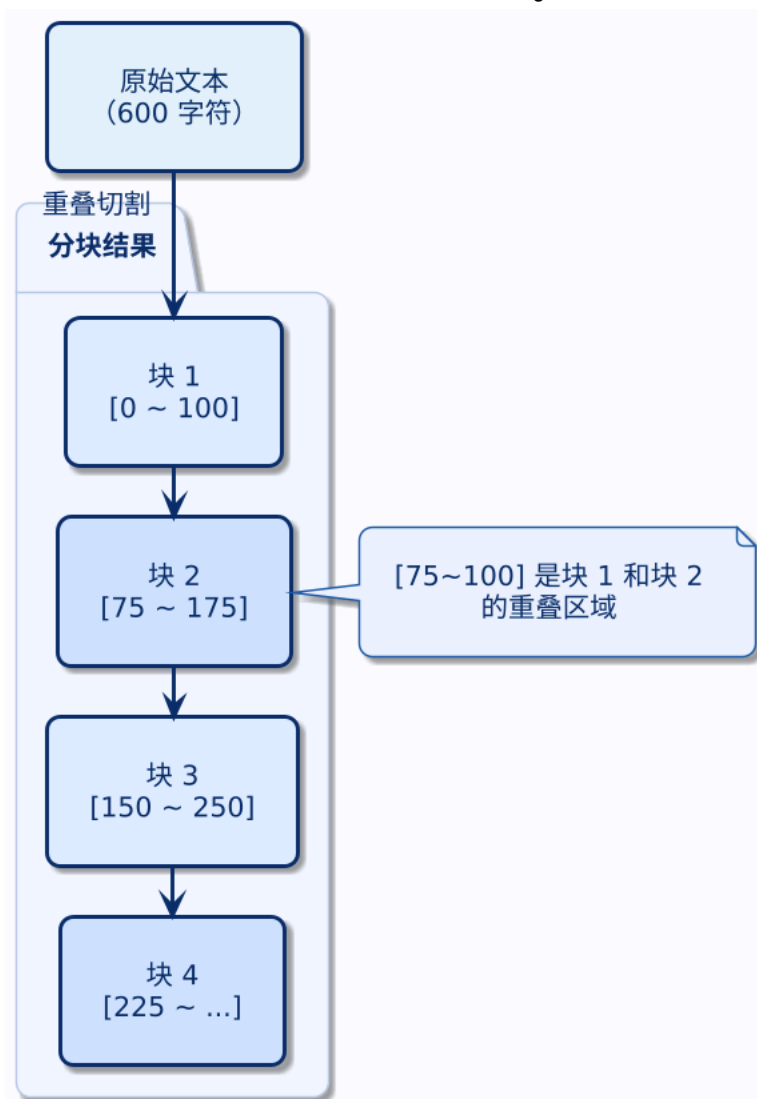
2.1 原理

重叠分块是对固定大小分块的直接改进。核心思路很简单：切块的时候，相邻两个块之间留一段重叠区域，这样即使切割点落在句子中间，重叠部分也能保证关键信息不会被完全切断。

还是用退货政策那段文本，chunkSize = 100，overlap = 25：

- 块 1：从第 1 个字符开始，取 100 个字符
- 块 2：从第 75 个字符开始（100-25=75），取 100 个字符
- 块 3：从第 150 个字符开始，取 100 个字符
-

用图来看就是这样：



和固定大小分块相比，重叠分块多了一个 `overlap` 参数，但效果提升明显——边界处的信息不再完全丢失。

2.2 Java 代码实现

```
import java.util.ArrayList;
import java.util.List;

public class OverlappingChunker {

    /**
     * 重叠分块——在固定大小的基础上，相邻块之间保留重叠区域
     */
    public static List<String> chunk(String text, int chunkSize, int overlap) {
        if (overlap >= chunkSize) {
            throw new IllegalArgumentException("overlap 必须小于 chunkSize");
        }
        List<String> chunks = new ArrayList<>();
        int step = chunkSize - overlap;
        int start = 0;
        while (start < text.length()) {
            int end = Math.min(start + chunkSize, text.length());
            chunks.add(text.substring(start, end));
            start += step;
        }
        return chunks;
    }

    public static void main(String[] args) {
        String text = "自签收之日起 7 天内，商品未经使用且不影响二次销售的，"
            + "消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等"
```

+ "特殊品类不适用此规则，具体以商品详情页标注为准。"

+ "退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。";

```
List<String> chunks = chunk(text, 40, 10);

for (int i = 0; i < chunks.size(); i++) {
    System.out.println("=== 块 " + (i + 1) + " ===");
    System.out.println(chunks.get(i));
    System.out.println();
}
}
```

对比固定大小分块的代码，核心区别就一行：`start += step` 而不是 `start = end`。`step = chunkSize - overlap`，每次前进的步长比块大小小一点，这样相邻块就自然产生了重叠。

2.3 优缺点

维度	说明
优点	实现简单，有效缓解边界处的语义断裂问题
缺点	仍然不看文本内容，只是用重叠来弥补；overlap 会导致存储量增加
适合	大多数通用场景的入门方案，尤其是你还没确定用什么策略的时候
不适合	对语义完整性要求很高的场景，比如法律条款、合同文本

3. 递归分块 (Recursive Chunking)

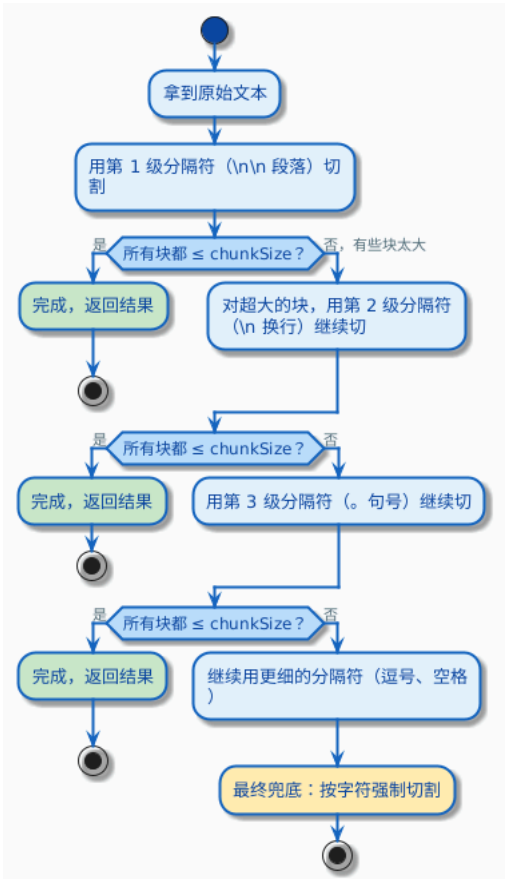
3.1 原理

递归分块是目前实践中最常用的策略。它的思路可以用一句话概括：先尝试用最大的分隔符切，切完如果某个块还是太大，就换一个更小的分隔符继续切，直到所有块都在 `chunkSize` 以内。

具体来说，它维护一个分隔符列表，按优先级从高到低排列，比如：

```
["\n\n", "\n", "。", "，", "，", " " , ""]
```

切割过程是这样的：



这个先粗后细的过程就是递归的含义——不是一刀切到底，而是逐层细化。

为什么这种方式好？因为它尽最大努力保留文本的结构。能按段落切就按段落切，段落太长了才按句子切，句子还太长才按逗号切……只有在万不得已的时候才会像固定大小分块那样按字符硬切。

拿我们的电商知识库来说，如果 chunkSize 设成 200，递归分块会先尝试按章节（空行）切割。退货政策那一段大约 150 字，没超过 200，就完整保留为一个块。如果某个章节特别长超过了 200 字，才会进一步按句号切成更小的块。

3.2 优缺点

维度	说明
优点	兼顾了语义完整性和块大小控制，是目前最通用的分块策略
缺点	分隔符列表需要根据语言调整（中文和英文的标点不同）；依赖文本中存在合理的分隔符
适合	绝大多数场景，尤其是你不确定该用什么策略的时候，递归分块是最安全的默认选择
不适合	对分块有特殊要求的场景，比如代码文件（需要按函数/类来切）、表格数据（需要按行来切）

4. 语义分块 (Semantic Chunking)

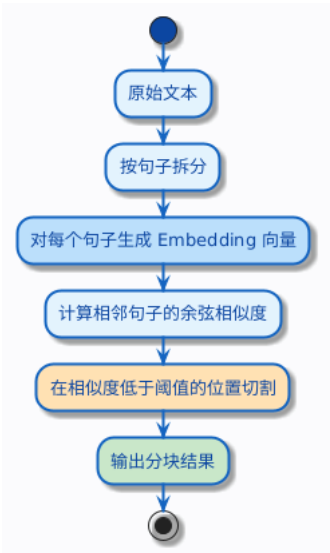
4.1 原理

前面三种策略有一个共同的局限：它们都是基于规则的——要么按字数切，要么按标点符号切。它们不理解文本在说什么。

语义分块换了一个完全不同的思路：用 Embedding 模型来判断文本的语义相似度，在语义发生明显变化的地方切割。

具体过程是这样的：

- 1. 先把文本按句子拆开（这一步可以简单地按句号切）
- 2. 对每个句子生成一个向量 (Embedding)
- 3. 计算相邻句子之间的向量相似度
- 4. 当相邻句子的相似度低于某个阈值时，说明话题发生了转换，在这里切一刀



还是用电商知识库的例子。退货政策那一段里，前两句都在讲退货条件，它们的语义向量会比较接近，相似度高。但从退货政策的最后一句过渡到换货政策的第一句时，话题发生了明显变化，两个句子的向量相似度会骤降——语义分块就在这个位置切一刀。

这种方式的好处是显而易见的：它切出来的每个块在语义上是高度内聚的，不会出现一个块里混着两个不相关话题的情况。

相比 Embedding 计算相似度的方式，直接让大模型来做分块决策更加直观——大模型本身就具备强大的语义理解能力，可以直接读懂文本，判断哪里应该切分。

最直接的思路：把文本交给大模型，让它找出主题切换的位置。

对比 Embedding vs LLM 分块：

维度	Embedding 语义分块	LLM 辅助分块
原理	计算相邻句子的向量相似度	大模型直接理解文本语义
分块质量	依赖 Embedding 模型质量	通常更准确，能处理复杂语境
速度	快（毫秒级）	慢（秒级）
成本	低	高
适用场景	大批量文档处理	高价值文档、需要精细分块

4.2 优缺点

维度	说明
优点	切割点基于语义而非规则，分块质量最高，每个块的主题高度内聚
缺点	需要调用 Embedding 或者 Chat 模型，有额外的计算成本和延迟；阈值需要调参；对模型的质量有依赖
适合	对检索精度要求很高的场景，比如法律文档问答、医疗知识库、金融合规文档
不适合	文档量特别大且对延迟敏感的场景；文本本身结构已经很清晰的场景（用递归分块就够了，没必要上语义分块）

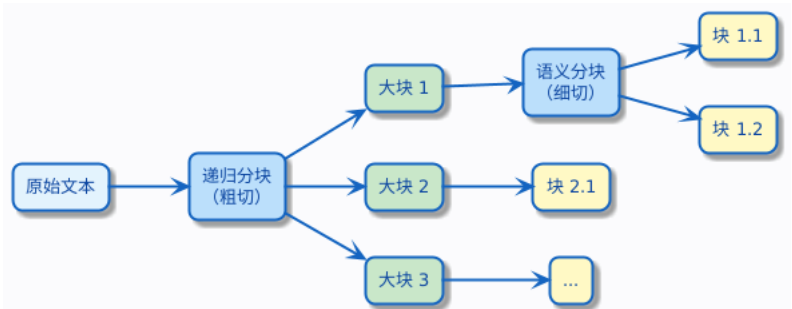
5. 混合分块 (Hybrid Chunking)

5.1 原理

实际项目中，单一的分块策略往往不够用。不同类型的文档、甚至同一份文档的不同部分，可能适合不同的分块方式。混合分块的思路就是：把多种策略组合起来用，取长补短。

常见的组合方式有几种：

第一种，递归分块 + 语义分块。先用递归分块做粗切，把文本按段落、章节切成大块；然后对每个大块再用语义分块做细切，确保每个最终的块在语义上是内聚的。



第二种，按文档类型选策略。比如在一个企业知识库系统中，产品手册用递归分块，FAQ 用按问答对切割，合同文本用语义分块。在代码层面，就是一个路由逻辑，根据文档的类型或来源选择不同的分块器。

第三种，分块 + 后处理。先用递归分块切完，然后对结果做一轮后处理：合并太短的块、拆分太长的块、给每个块补充元数据（比如所属章节标题、文档来源）。

拿我们的电商知识库来说，一个实际的混合方案可能是这样的：

知识库正文（退货政策、物流规则等）：用递归分块，按章节切割

FAQ 问答对：按问答对切割，每个 Q&A 是一个块

商品详情页：先提取结构化字段（价格、规格），再对描述文本用重叠分块

5.2 优缺点

维度	说明
优点	灵活，能针对不同内容选择最合适的策略，整体效果最好
缺点	实现复杂度高，需要维护多套分块逻辑和路由规则
适合	企业级 RAG 系统，文档类型多样，对检索质量有较高要求
不适合	简单的 demo 或 POC 阶段，杀鸡用牛刀

用 ChunkViz 直观对比分块效果

光看文字描述可能还不够直观。推荐打开 [ChunkViz](#) 这个在线工具，它可以可视化地展示不同分块策略的效果。

操作步骤：

1. 打开 <https://chunkviz.up.railway.app/>
2. 在文本框中粘贴我们的示例文本（或者任何你想测试的文本）
3. 在右侧选择分块策略：

CharacterTextSplitter：对应固定大小分块

RecursiveCharacterTextSplitter：对应递归分块
4. 调整 Chunk Size 和 Chunk Overlap 的滑块
5. 页面会实时用不同颜色高亮显示每个块的范围

你可以做这几组对比实验：

实验	策略	chunkSize	overlap	观察重点
实验 1	CharacterTextSplitter	100	0	看看块的边界是不是经常切在句子中间
实验 2	CharacterTextSplitter	100	20	对比实验 1，看 overlap 区域（颜色重叠的部分）
实验 3	RecursiveCharacterTextSplitter	100	0	对比实验 2，看递归策略是不是更倾向于在段落/句子边界切割
实验 4	RecursiveCharacterTextSplitter	200	0	增大 chunkSize，看块的数量和完整性变化

通过这几组实验，你应该能非常直观地感受到：递归分块在保持语义完整性方面明显优于固定大小分块。

语言模型有上下文窗口。这是它们单次处理文本的长度。

尽管上下文长度在不断增加，但研究表明，当语言模型被给予更少（但更相关）的信息时，它们在任务上的表现会得到提升。

但是，你该如何选择相关的数据子集呢？当人工手动操作时，这很简单，但事实证明，让计算机执行这一任务非常困难。

一种常见的方法是将大型数据通过分块或子集的方式拆分成更小的片段。为了实现这一点，你需要选择一个分块策略。

选择上方的不同分块策略，看看它们如何影响文本，如果你愿意，可以添加自己的文本。

你会看到代表不同块的不同颜色。这可能是一个块 1。这可能是一个块 2。有时一个块会在句子中间发生变化（这不太好）。如果任何块有重叠的文本，那些会以橙色显示。

块大小：你末尾块的长度的字符数

块重叠（绿色）：连续块共享的重叠或交叉的量

注意：*文本分割器会修剪 js、python 和 markdown 分割器末尾的空白，这就是为什么文本会跳来跳去，*重叠锁定为块大小的 50%以下 *使用简单的分析（注重隐私）来了解我的托管账单。

对于文本分割器的实现，请查看 LangChain [\(py, js\)](#) 和 Llama Index [\(py, js\)](#)

MIT 许可证，[开源](#)，欢迎 PR

大家可以看下，粉色、蓝色、荧光绿色以及黄色等，可能都是不同分块的颜色，但墨绿色是固定重叠的标识。

1. CharacterTextSplitter

测试字符分割器配置：

Chunk Size: 100

Chunk Overlap: 20

一、退货政策

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在审核通过后 3 个工作日内寄出，届时会通过短信通知物流单号。

三、会员权益

分割器：

字符分割器

块大小：

100

块重叠：

20

总字符数：770

块数量：8

平均块大小：96.3

上传 .txt

一、退货政策

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在审核通过后 3 个工作日内寄出，届时会通过短信通知物流单号。

三、会员权益

普通会员享受 9.5 折优惠，每月可领取 2 张满 200 减 20 的优惠券。黄金会员享受 9 折优惠，每月可领取 4 张满 200 减 30 的优惠券，且退换货享受免运费服务。钻石会员享受 8.5 折优惠，每月可领取 6 张满 200 减 50 的优惠券，退换货免运费，并享有专属客服通道。

四、物流配送

默认使用顺丰快递，偏远地区使用邮政 EMS。订单金额满 99 元包邮，未满 99 元收取 8 元运费。大件商品（家具、家电等）使用德邦物流，配送费根据商品重量和收货地址单独计算。预计配送时间：一线城市 1-2 天，二三线城市 2-4 天，偏远地区 5-7 天。

五、售后服务

所有商品享受 1 年质保服务。质保期内出现非人为损坏的质量问题，可免费维修或更换。超出质保期的维修服务按成本价收取配件费和人工费。如需售后服务，请拨打客服热线 400-XXX-XXXX 或在 APP 内提交售后工单。

可以看到，经过切割后，文本原始语义被切分相对严重，因此在实际使用过程中需要不断尝试调整才能获得最优效果。

2. RecursiveCharacterTextSplitter

测试递归字符分割器配置：

Chunk Size: 200

Chunk Overlap: 0

https://articles.zsxq.com/id_kdbpwwl7vfft.html

11/15

一、退货政策

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在审核通过后 3 个工作日内寄出，届时会通过短信通知物流单号。

三、会员权益

上传 .txt

分割器：递归字符文本分割器

块大小：200

块重叠：0

总字符数：770

块数量：8

平均块大小：96.3

一、退货政策

自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在审核通过后 3 个工作日内寄出，届时会通过短信通知物流单号。

三、会员权益

普通会员享受 9.5 折优惠，每月可领取 2 张满 200 减 20 的优惠券。黄金会员享受 9 折优惠，每月可领取 4 张满 200 减 30 的优惠券，且退换货享受免运费服务。钻石会员享受 8.5 折优惠，每月可领取 6 张满 200 减 50 的优惠券，退换货免运费，并享有专属客服通道。

四、物流配送

默认使用顺丰快递，偏远地区使用邮政 EMS。订单金额满 99 元包邮，未满 99 元收取 8 元运费。大件商品（家具、家电等）使用德邦物流，配送费根据商品重量和收货地址单独计算。预计配送时间：一线城市 1-2 天，二三线城市 2-4 天，偏远地区 5-7 天。

五、售后服务

所有商品享受 1 年质保服务。质保期内出现非人为损坏的质量问题，可免费维修或更换。超出质保期的维修服务按成本价收取配件费和人工费。如需售后服务，请拨打客服热线 400-XXX-XXXX 或在 APP 内提交售后工单。

在格式较为固定的文本中（例如标题后紧跟一句说明，或结构简单的 Q&A 块），递归字符分割通常能取得较好的效果。但当问答结构更复杂、语义跨度更大时，边界判定仍可能不够准确，导致切分偏差。

比如在退货政策这种场景里，同一个条款下面往往会包含好几句话甚至多个条件，这时递归字符分割就不太好用了，容易把本应属于同一语义单元的内容切散。

一、退货政策

第一句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

第二句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则.....

第三句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则.....

上传 .txt

分割器：递归字符文本分割器

块大小：200

块重叠：0

总字符数：770

块数量：8

平均块大小：96.3

一、退货政策

第一句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则，具体以商品详情页标注为准。退货运费由消费者承担，如因商品质量问题退货，运费由商家承担。

第二句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则.....

第三句话：自签收之日起 7 天内，商品未经使用且不影响二次销售的，消费者可申请七天无理由退货。生鲜食品、定制商品、贴身衣物等特殊品类不适用此规则.....

二、换货政策

自签收之日起 15 天内，商品存在质量问题的，消费者可申请免费换货。换货时需提供订单编号、商品照片及问题描述。换货商品将在审核通过后 3 个工作日内寄出，届时会通过短信通知物流单号。

整体来看，递归分块属于较基础的切分方法，更适用于轻量、结构明确的应用场景。

分块策略怎么选：一张表帮你决定

1. 不同文档类型的推荐策略

文档类型	推荐策略	理由
产品手册 / 知识库	递归分块	有清晰的章节、段落结构，递归策略能很好地利用这些结构
FAQ / 问答对	递归分块	每个 Q&A 是一个自然单元，不应该被拆开
合同 / 法律文档	语义分块	条款之间的边界需要精确识别，规则分块容易切错
日志文件	固定大小分块或按行切割	日志通常每行一条记录，结构简单
代码文件	专用的代码分块器（按函数/类切）	通用的文本分块策略不适合代码，需要理解代码结构
HTML 页面	先清洗 HTML 标签，再用递归分块	HTML 标签会干扰分块，需要先处理
格式混乱的文本（OCR 等）	重叠分块	没有可靠的分隔符可用，重叠至少能缓解边界断裂
多类型混合的企业知识库	混合分块	不同类型的文档用不同策略，效果最好

2. chunkSize 和 overlap 的经验值参考

这些不是标准答案，而是社区实践中比较常见的起始值，你可以在此基础上根据实际效果调整：

参数	推荐范围	说明
chunkSize	200 ~ 1000 字符	问答场景偏小（200500），摘要场景偏大（5001000）
overlap	chunkSize 的 10% ~ 25%	比如 chunkSize=500 时，overlap 设 50~125

一个实用的调参思路：从 chunkSize=500、overlap=50 开始，跑几个测试 query 看检索效果，如果发现检索结果不够精准就调小 chunkSize，如果发现上下文经常断裂就调大 overlap。

3. 企业级解决方案

真实项目里，分块效果不好，往往不只是 chunkSize 没调对，还有可能是上游抽取出来的文本就已经“脏”了——尤其是 PDF 这种格式：

- 页眉页脚、目录、页码混进正文，语义被大量噪音稀释
- 断行/连字把一句话拆成多段，导致按段落/句子递归分割失效
- 表格被打散成碎词或错位字段，检索命中但无法形成可读上下文

所以通常不会“抽完就切”。更稳妥的流程是：**先用Tika等工具做文本抽取→再用清洗器做结构修复与去噪→最后才进入分块与向量化**。分块层面也有类似的现实问题。像 Dify、RAGFlow 这类标准 RAG 方案，很多时候依赖单一分块策略，调参就容易出现“顾此失彼”：chunkSize 调大，A 类问题更容易召回完整上下文，但 B 类问题的检索精度可能下降；chunkSize 调小，B 类问题更精准了，A 类问题又容易被切断。针对**中小规模文档**，更实用的做法是：先用基础策略把文档拆成初稿块，然后在此基础上做一轮**人工二次编排**——根据相邻块的语义关系进行补齐、合并或重分配，让最终的 chunk 更贴合真实问题类型。

首页 / 知识库管理 / 文档管理 / 切片管理

分块管理

基本信息.md (知识库: 2018586042835763200)

返回文档

重建向量

+ 新建分块

Chunk 列表

支持编辑、启停、批量操作

全部状态

刷新

批量启用

批量禁用

全量启用

全量禁用

☐	序号	内容	状态	字符数	Token数	更新时间	操作
☐	0	🍏 蒂姆·库克 (Tim Cook) 个人介绍 **Tim Cook** 是美国商业高管，现任 **Apple (苹果公司)** 首席执行官 (CEO)。他以稳健务实的管理风格、对供应链与运营的深度理解，以及对隐私、环保和社会责任的重视而闻名。...	启用	506	323	2026/2/3 23:54:24	编辑 禁用 删除
☐	1	le Music、iCloud 等)，并持续推动公司在可再生能源、用户隐私保护和无障碍设计等方面的投入。在他的领导下，苹果从一家以硬件为核心的科技公司，逐步发展为“硬件 + 软件 + 服务”并重的综合生态企业，市值多次刷新全球纪录。...	启用	472	344	2026/2/3 23:54:24	编辑 禁用 删除
☐	2	更偏向数据驱动与长期规划 - **重视价值观**：公开支持隐私保护、环境可持续和多元包容 - **运营型领袖**：擅长把复杂系统拆解为可执行流程 - **公开出柜的 CEO**：是全球最具影响力的公开同性恋企业领袖之一 ----- #...	启用	364	285	2026/2/3 23:54:24	编辑 禁用 删除
☐	3	在库克时代，苹果还多了一层含义： **用成熟的商业体系去长期影响世界**。 ----- 如果你愿意，我也可以继续帮你整理： - 📌 他任内最重要的苹果产品与战略变化 - 🌍 与其他科技 CEO 的风格对比 - 🍏 他对苹果未来方...	启用	136	101	2026/2/3 23:54:24	编辑 禁用 删除

共 4 条

上一页

1 / 1

下一页

可以在列表中点击某个具体分块进入编辑，对内容进行调整；保存后，底层的 Chunk 会同步更新并生效。

编辑分块

×

手动维护分块内容

内容

🍏 蒂姆·库克 (Tim Cook) 个人介绍

Tim Cook 是美国商业高管，现任 **Apple (苹果公司)** 首席执行官 (CEO)。他以稳健务实的管理风格、对供应链与运营的深度理解，以及对隐私、环保和社会责任的重视而闻名。

基本信息

- **中文名**：蒂姆·库克

- **英文名**：Tim Cook

- **出生日期**：1960 年 11 月 1 日

- **年龄**：65 岁 (截至 2026 年初)

- **国籍**：美国

- **现任职务**：苹果公司 CEO (自 2011 年起)

他是谁？为什么重要？

蒂姆·库克在 2011 年正式接任苹果 CEO，此前长期负责公司的全球运营与供应链体系。他上任后，苹果在保持产品创新的同时，大幅强化了服务业务 (如 App Store、Apple Music、iCloud 等)，并持续推动公司在可

取消

保存

文末总结

分块是 RAG 数据准备阶段的关键一步。拿到 Tika 提取的纯文本后，不能直接用，需要切成大小合适、语义完整的小段，才能在后续的检索环节中被精准匹配到。

五种分块策略各有定位：固定大小分块最简单但最粗暴；重叠分块用重叠区域缓解边界断裂；递归分块逐层细化，是大多数场景的默认选择；语义分块基于 Embedding 做语义级切割，精度最高但成本也最高；混合分块组合多种策略，适合企业级复杂场景。

chunkSize 和 overlap 这两个参数没有最优值，需要根据你的文档类型和检索场景来调。记住起步参考值：chunkSize=500、overlap=50，然后根据实际效果微调。

分块之后，每个文本块还只是一段人类能读懂的文字。计算机要做相似度检索，需要把这些文字转成它能理解的数字表示——这就是向量化 (Embedding) 。

